```
/**
 * WA Trading Intelligence Platform — Full Production Backend
 * MongoDB + JWT Auth + AI Scoring + Team Features + Price Alerts
 */

const express = require('express');
const cors = require('cors');
const path = require('path');
const crypto = require('crypto');
const app = express();

app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

// ── ENV ──
const CLAUDE_KEY  = process.env.CLAUDE_API_KEY || '';
const MONGO_URI   = process.env.MONGO_URI || '';
const JWT_SECRET  = process.env.JWT_SECRET || 'wa-trading-secret-2024';
const PORT        = process.env.PORT || 3000;

// ── IN-MEMORY FALLBACK (used if no MongoDB) ──
const mem = {
  users: [],
  listings: [],
  traders: {},
  deals: {},
  notes: {},
  alerts: [],
  snapshots: [],
  rawMessages: [],
};

// ── MONGODB (optional, graceful fallback) ──
let db = null;
async function connectMongo() {
  if (!MONGO_URI) { console.log('⚠️  No MONGO_URI — using in-memory storage'); re
  try {
    const { MongoClient } = require('mongodb');
    const client = new MongoClient(MONGO_URI);
    await client.connect();
    db = client.db('wa_trading');
    console.log('✅ MongoDB connected');
    // Indexes
```

```
    await db.collection('listings').createIndex({ timestamp: -1 });
    await db.collection('listings').createIndex({ model: 1 });
    await db.collection('listings').createIndex({ type: 1 });
    await db.collection('users').createIndex({ email: 1 }, { unique: true });
  } catch (e) {
    console.error('MongoDB error:', e.message);
    console.log('⚠️  Falling back to in-memory storage');
  }
}

// ── DB HELPERS ──
const DB = {
  async getListings(filter={}, limit=200, skip=0) {
    if (db) return db.collection('listings').find(filter).sort({ timestamp: -1 })
    let list = mem.listings;
    if (filter.type) list = list.filter(l => l.type === filter.type);
    if (filter.timestamp?.$gt) list = list.filter(l => l.timestamp > filter.times
    return list.slice(skip, skip + limit);
  },
  async insertListing(l) {
    if (db) return db.collection('listings').insertOne(l);
    mem.listings.unshift(l);
    if (mem.listings.length > 5000) mem.listings.pop();
  },
  async countListings(filter={}) {
    if (db) return db.collection('listings').countDocuments(filter);
    return mem.listings.filter(l => (!filter.type || l.type === filter.type)).len
  },
  async getUser(email) {
    if (db) return db.collection('users').findOne({ email });
    return mem.users.find(u => u.email === email);
  },
  async insertUser(u) {
    if (db) return db.collection('users').insertOne(u);
    mem.users.push(u);
  },
  async getUsers() {
    if (db) return db.collection('users').find({}).toArray();
    return mem.users;
  },
  async updateListing(id, update) {
    if (db) return db.collection('listings').updateOne({ id }, { $set: update });
    const l = mem.listings.find(x => x.id === id);
    if (l) Object.assign(l, update);
  },
  async saveSnapshot(snap) {
    if (db) return db.collection('snapshots').insertOne(snap);
```

```
    mem.snapshots.push(snap);
    if (mem.snapshots.length > 100) mem.snapshots.shift();
  },
  async getSnapshots(limit=30) {
    if (db) return db.collection('snapshots').find({}).sort({ date: -1 }).limit(l
    return mem.snapshots.slice(-limit).reverse();
  },
  async saveAlert(alert) {
    if (db) return db.collection('alerts').insertOne(alert);
    mem.alerts.push(alert);
  },
  async getAlerts(email) {
    if (db) return db.collection('alerts').find({ email, active: true }).toArray(
    return mem.alerts.filter(a => a.email === email && a.active);
  },
};

// ── MULTI-CHANNEL STORE ──
// Each WhatsApp number is a "channel" with its own token and group settings
const channels = {};
// channels[channelId] = { id, label, token, active, groups: { groupId: { name, e

function getOrCreateChannel(channelId, label='') {
  if (!channels[channelId]) {
    channels[channelId] = { id: channelId, label: label||channelId, token: '', ac
  }
  return channels[channelId];
}

function isGroupAllowed(channelId, groupId) {
  const ch = channels[channelId];
  if (!ch) return true; // unknown channel — allow
  const grp = ch.groups[groupId];
  if (!grp) return true; // new group not yet configured — allow by default
  return grp.enabled !== false; // default allow
}

function trackGroup(channelId, groupId, groupName, msgCount=1) {
  const ch = getOrCreateChannel(channelId);
  if (!ch.groups[groupId]) {
    ch.groups[groupId] = { id: groupId, name: groupName||groupId, enabled: true,
  }
  ch.groups[groupId].messageCount += msgCount;
  ch.groups[groupId].lastSeen = new Date();
  if (groupName && !ch.groups[groupId].name) ch.groups[groupId].name = groupName;
}
```

```
// ── PHONE → COUNTRY ──
const PREFIXES = {
  '971':'UAE','966':'Saudi Arabia','965':'Kuwait','973':'Bahrain',
  '974':'Qatar','968':'Oman','962':'Jordan','961':'Lebanon',
  '963':'Syria','964':'Iraq','20':'Egypt','212':'Morocco',
  '213':'Algeria','216':'Tunisia','249':'Sudan','92':'Pakistan',
  '91':'India','880':'Bangladesh','94':'Sri Lanka','44':'UK',
  '1':'USA','49':'Germany','33':'France','90':'Turkey',
};
function phoneToCountry(phone) {
  const d = phone.replace(/\D/g, '');
  for (const len of [3,2,1]) { const p = d.slice(0,len); if (PREFIXES[p]) return
  return null;
}

// ── JWT ──
function makeToken(user) {
  const payload = Buffer.from(JSON.stringify({ email: user.email, name: user.name
  const sig = crypto.createHmac('sha256', JWT_SECRET).update(payload).digest('hex
  return `${payload}.${sig}`;
}
function verifyToken(token) {
  try {
    const [payload, sig] = token.split('.');
    const expected = crypto.createHmac('sha256', JWT_SECRET).update(payload).dige
    if (sig !== expected) return null;
    const data = JSON.parse(Buffer.from(payload, 'base64').toString());
    if (data.exp < Date.now()) return null;
    return data;
  } catch { return null; }
}
function authMiddleware(req, res, next) {
  const token = req.headers.authorization?.replace('Bearer ', '');
  if (!token) return res.status(401).json({ error: 'No token' });
  const user = verifyToken(token);
  if (!user) return res.status(401).json({ error: 'Invalid token' });
  req.user = user;
  next();
}

// ── HASH ──
function hashPw(pw) { return crypto.createHash('sha256').update(pw + JWT_SECRET).

// ── SERVE APP ──
app.get('/', (req, res) => res.sendFile(path.join(__dirname, 'public', 'index.htm
app.get('/health', async (req, res) => {
  const count = await DB.countListings();
```

```
  res.json({ status: 'ok', listings: count, db: db ? 'mongodb' : 'memory' });
});

// ── AUTH ROUTES ──
app.post('/api/auth/register', async (req, res) => {
  try {
    const { email, password, name, company } = req.body;
    if (!email || !password || !name) return res.status(400).json({ error: 'Missi
    const existing = await DB.getUser(email);
    if (existing) return res.status(400).json({ error: 'Email already registered'
    const users = await DB.getUsers();
    const role = users.length === 0 ? 'admin' : 'member';
    const user = { email, name, company: company||'', role, passwordHash: hashPw(
    await DB.insertUser(user);
    res.json({ token: makeToken(user), user: { email, name, role, company } });
  } catch (e) { res.status(500).json({ error: e.message }); }
});

app.post('/api/auth/login', async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await DB.getUser(email);
    if (!user || user.passwordHash !== hashPw(password)) return res.status(401).j
    res.json({ token: makeToken(user), user: { email: user.email, name: user.name
  } catch (e) { res.status(500).json({ error: e.message }); }
});

app.get('/api/auth/me', authMiddleware, (req, res) => res.json(req.user));

app.get('/api/team', authMiddleware, async (req, res) => {
  const users = await DB.getUsers();
  res.json(users.map(u => ({ email: u.email, name: u.name, role: u.role, company:
});

// ── WHAPI WEBHOOK ──
app.post('/webhook', async (req, res) => {
  res.sendStatus(200);
  // Channel identified by query param: /webhook?channel=MY_CHANNEL_ID
  // or by X-Channel-Id header
  const channelId = req.query.channel || req.headers['x-channel-id'] || 'default'
  const messages = req.body.messages || [];
  for (const msg of messages) {
    const text = msg.text?.body || msg.caption || '';
    if (!text || text.length < 8 || msg.from_me) continue;
    const sender = (msg.from || 'Unknown').replace('@s.whatsapp.net','').replace(
    const group  = msg.chat_id || 'Unknown';
    const groupName = msg.chat_name || msg.notify || group;
```

```
const ts     = msg.timestamp || Math.floor(Date.now()/1000);

// Track group in channel registry
trackGroup(channelId, group, groupName);

// Skip if group is disabled
if (!isGroupAllowed(channelId, group)) continue;

mem.rawMessages.unshift({ text, sender, group, groupName, channelId, ts });
if (mem.rawMessages.length > 200) mem.rawMessages.pop();
const lower = text.toLowerCase();
const isTrading = ['wts','wtb','selling','buying','for sale','looking for','a
if (!isTrading) continue;
const classified = await classifyMessage(text, sender);
if (!classified || classified.type === 'UNKNOWN') continue;
const sellerCountry = phoneToCountry(sender);
const listing = {
  id: `${ts}-${Math.random().toString(36).slice(2,7)}`,
  type: classified.type,
  condition: classified.condition,
  model: classified.model || null,
  storage: classified.storage || null,
  ram: classified.ram || null,
  color: classified.color || null,
  qty: classified.qty || null,
  grade: classified.condition === 'Used' ? (classified.grade || null) : null,
  price: classified.price || null,
  currency: classified.currency || 'USD',
  summary: classified.summary || text.slice(0,80),
  dealScore: null,
  isFlagged: false,
  flagReason: null,
  sender,
  channelId,
  sellerCountry,
  listingCountry: classified.country || null,
  group,
  timestamp: ts,
  raw: text,
  assignedTo: null,
  status: 'new',
  notes: [],
  createdAt: new Date(),
};
// AI deal scoring (async, don't block)
scoreListing(listing).then(scored => {
  Object.assign(listing, scored);
```

```
      DB.updateListing(listing.id, scored);
    });
    await DB.insertListing(listing);
    // Check price alerts
    checkAlerts(listing);
    // Update trader
    if (!mem.traders[sender]) mem.traders[sender] = { wts:0, wtb:0, country: sell
    if (listing.type === 'WTS') mem.traders[sender].wts++; else mem.traders[sende
  }
});

// ── AI CLASSIFICATION ──
async function classifyMessage(text, sender) {
  if (!CLAUDE_KEY) return fallback(text);
  try {
    const r = await fetch('https://api.anthropic.com/v1/messages', {
      method: 'POST',
      headers: { 'Content-Type':'application/json', 'x-api-key':CLAUDE_KEY, 'anth
      body: JSON.stringify({
        model: 'claude-sonnet-4-20250514', max_tokens: 400,
        system: `Electronics trading WhatsApp classifier for Middle East markets.
{"type":"WTS"|"WTB"|"UNKNOWN","condition":"Brand New"|"Used","model":"brand+model
        messages: [{ role:'user', content:`Phone:${sender}\nMessage:${text}` }],
      }),
    });
    const d = await r.json();
    return JSON.parse((d.content?.[0]?.text||'{}').trim());
  } catch { return fallback(text); }
}

// ── AI DEAL SCORING ──
async function scoreListing(listing) {
  if (!CLAUDE_KEY || !listing.model) return { dealScore: null, isFlagged: false }
  try {
    const allListings = await DB.getListings({ model: listing.model }, 20);
    const prices = allListings.filter(l=>l.price&&l.type===listing.type).map(l=>l
    const avgPrice = prices.length ? Math.round(prices.reduce((a,b)=>a+b,0)/price
    const r = await fetch('https://api.anthropic.com/v1/messages', {
      method: 'POST',
      headers: { 'Content-Type':'application/json', 'x-api-key':CLAUDE_KEY, 'anth
      body: JSON.stringify({
        model: 'claude-sonnet-4-20250514', max_tokens: 200,
        system: `You score electronics trading listings 1-10 and detect fake/susp
        messages: [{ role:'user', content:`Listing: ${listing.type} ${listing.con
      }),
    });
    const d = await r.json();
```

```
    const result = JSON.parse((d.content?.[0]?.text||'{}').trim());
    return { dealScore: result.score||null, isFlagged: result.isFlagged||false, f
  } catch { return { dealScore: null, isFlagged: false }; }
}

// ── FAKE DETECTOR ──
async function detectFake(text, price, model) {
  if (!CLAUDE_KEY) return { isFake: false };
  try {
    const r = await fetch('https://api.anthropic.com/v1/messages', {
      method: 'POST',
      headers: { 'Content-Type':'application/json', 'x-api-key':CLAUDE_KEY, 'anth
      body: JSON.stringify({
        model: 'claude-sonnet-4-20250514', max_tokens: 150,
        system: 'Detect suspicious/fake electronics listings. Return ONLY JSON: {
        messages: [{ role:'user', content:`Model: ${model}\nPrice: $${price}\nMes
      }),
    });
    const d = await r.json();
    return JSON.parse((d.content?.[0]?.text||'{}').trim());
  } catch { return { isFake: false }; }
}

// ── PRICE ALERTS ──
async function checkAlerts(listing) {
  if (!listing.price || !listing.model) return;
  const alerts = mem.alerts.filter(a => a.active && a.model?.toLowerCase() === li
  alerts.forEach(a => { console.log(`🔔 Alert: ${a.email} — ${listing.model} at $
}

app.post('/api/alerts', authMiddleware, async (req, res) => {
  const { model, targetPrice } = req.body;
  if (!model || !targetPrice) return res.status(400).json({ error: 'Missing field
  const alert = { email: req.user.email, model, targetPrice: Number(targetPrice),
  await DB.saveAlert(alert);
  mem.alerts.push(alert);
  res.json({ ok: true, alert });
});

app.get('/api/alerts', authMiddleware, async (req, res) => {
  const alerts = await DB.getAlerts(req.user.email);
  res.json(alerts);
});

// ── LISTINGS API ──
app.get('/api/messages', authMiddleware, async (req, res) => {
  const since = parseInt(req.query.since) || 0;
```

```
  const limit  = parseInt(req.query.limit) || 100;
  const type   = req.query.type;
  const filter = { ...(since ? { timestamp: { $gt: since } } : {}), ...(type ? {
  const listings = await DB.getListings(filter, limit);
  res.json(listings);
});

app.get('/api/stats', authMiddleware, async (req, res) => {
  const all = await DB.getListings({}, 2000);
  const prices = all.filter(l=>l.price).map(l=>l.price);
  const avg = prices.length ? Math.round(prices.reduce((a,b)=>a+b,0)/prices.lengt
  const todayStart = new Date(); todayStart.setHours(0,0,0,0);
  const models={}, countries={}, grades={};
  all.forEach(l => {
    if(l.model) models[l.model]=(models[l.model]||0)+1;
    const c=l.sellerCountry||l.listingCountry; if(c) countries[c]=(countries[c]||
    if(l.grade) grades['Grade '+l.grade]=(grades['Grade '+l.grade]||0)+1;
  });
  res.json({
    total: all.length,
    wts: all.filter(l=>l.type==='WTS').length,
    wtb: all.filter(l=>l.type==='WTB').length,
    brandNew: all.filter(l=>l.condition==='Brand New').length,
    used: all.filter(l=>l.condition==='Used').length,
    avgPrice: avg,
    traders: Object.keys(mem.traders).length,
    today: all.filter(l=>l.timestamp*1000>todayStart.getTime()).length,
    hotCount: all.filter(l=>l.dealScore>=8).length,
    flagged: all.filter(l=>l.isFlagged).length,
    models: topN(models,8), countries: topN(countries,8), grades: topN(grades,6),
    topTraders: Object.entries(mem.traders).map(([name,d])=>({name,total:d.wts+d.
  });
});

// ── DEAL PIPELINE ──
app.patch('/api/listings/:id', authMiddleware, async (req, res) => {
  const { status, assignedTo, note } = req.body;
  const update = {};
  if (status) update.status = status;
  if (assignedTo) update.assignedTo = assignedTo;
  if (note) {
    if (!mem.notes[req.params.id]) mem.notes[req.params.id] = [];
    mem.notes[req.params.id].push({ text: note, by: req.user.name, at: new Date()
    update.notes = mem.notes[req.params.id];
  }
  await DB.updateListing(req.params.id, update);
  res.json({ ok: true });
```

```
});

// ── AI ROUTES ──
async function claudeCall(prompt, system, maxTokens=1000) {
  if (!CLAUDE_KEY) return '⚠️ No Claude API key configured on server.';
  const r = await fetch('https://api.anthropic.com/v1/messages', {
    method: 'POST',
    headers: { 'Content-Type':'application/json', 'x-api-key':CLAUDE_KEY, 'anthro
    body: JSON.stringify({ model:'claude-sonnet-4-20250514', max_tokens:maxTokens
  });
  const d = await r.json();
  return d.content?.[0]?.text || 'No response';
}

app.post('/api/ai/briefing', authMiddleware, async (req, res) => {
  const all = await DB.getListings({}, 200);
  const models={}, countries={};
  all.forEach(l=>{ if(l.model)models[l.model]=(models[l.model]||0)+1; const c=l.s
  const topModels = Object.entries(models).sort((a,b)=>b[1]-a[1]).slice(0,8).map(
  const wts=all.filter(l=>l.type==='WTS').length, wtb=all.filter(l=>l.type==='WTB
  const prices=all.filter(l=>l.price).map(l=>l.price);
  const avg=prices.length?Math.round(prices.reduce((a,b)=>a+b,0)/prices.length):0
  const result = await claudeCall(
    `Market data:\n- Listings: ${all.length} (${wts} WTS, ${wtb} WTB)\n- Avg pric
    'You are a senior Middle East electronics wholesale market analyst. Be concis
  );
  res.json({ result });
});

app.post('/api/ai/price', authMiddleware, async (req, res) => {
  const { model } = req.body;
  const listings = await DB.getListings({ model }, 30);
  const prices = listings.filter(l=>l.price).map(l=>`${l.type} ${l.condition} ${l
  const result = await claudeCall(
    `Device: ${model}\nOur market data:\n${prices||'No price data yet'}\n\nProvid
    'Expert Middle East electronics pricing analyst. Give specific USD and AED fi
  );
  res.json({ result });
});

app.post('/api/ai/profile', authMiddleware, async (req, res) => {
  const { query } = req.body;
  const all = await DB.getListings({}, 2000);
  const listings = all.filter(l=>(l.sender||'').includes(query)||(l.sender||'').t
  if (!listings.length) return res.json({ result: 'No listings found for: '+query
  const summary = listings.map(l=>`${l.type} ${l.condition} ${l.model||'?'} ${l.g
  const result = await claudeCall(
```

```
    `Trader: ${query}\nListings(${listings.length}): WTS:${listings.filter(l=>l.t
    'Electronics trader analyst. Be concise and analytical.'
  );
  res.json({ result });
});

app.post('/api/ai/autoreply', authMiddleware, async (req, res) => {
  const { listing, intent } = req.body;
  const result = await claudeCall(
    `Listing: ${listing.type} ${listing.condition} ${listing.model} ${listing.sto
    'You draft professional WhatsApp replies for electronics wholesale traders in
  );
  res.json({ result });
});

app.post('/api/ai/item', authMiddleware, async (req, res) => {
  const { question } = req.body;
  const result = await claudeCall(question, 'Expert in consumer electronics sold
  res.json({ result });
});

// ── MARGIN CALCULATOR ──
app.post('/api/margin', authMiddleware, async (req, res) => {
  const { buyPrice, sellPrice, qty, shippingCost, dutyPct } = req.body;
  const buy = Number(buyPrice), sell = Number(sellPrice), q = Number(qty)||1;
  const shipping = Number(shippingCost)||0, duty = Number(dutyPct)||0;
  const totalCost = (buy + shipping/q) * (1 + duty/100);
  const profit = sell - totalCost;
  const margin = ((profit/sell)*100).toFixed(1);
  const totalProfit = profit * q;
  res.json({ totalCost: totalCost.toFixed(2), profit: profit.toFixed(2), margin,
});

// ── SNAPSHOTS (weekly auto-save) ──
app.post('/api/snapshots', authMiddleware, async (req, res) => {
  const all = await DB.getListings({}, 2000);
  const models={};
  all.forEach(l=>{ if(l.model)models[l.model]=(models[l.model]||0)+1; });
  const snap = { date: new Date(), totalListings: all.length, wts: all.filter(l=>
  await DB.saveSnapshot(snap);
  res.json({ ok: true, snap });
});

app.get('/api/snapshots', authMiddleware, async (req, res) => {
  const snaps = await DB.getSnapshots(30);
  res.json(snaps);
});
```

```
// ── EXPORT REPORT ──
app.get('/api/report', authMiddleware, async (req, res) => {
  const all = await DB.getListings({}, 2000);
  const models={}, countries={};
  all.forEach(l=>{ if(l.model)models[l.model]=(models[l.model]||0)+1; const c=l.s
  res.json({ listings: all, models: topN(models,20), countries: topN(countries,20
});

// ── FALLBACK CLASSIFIER ──
function fallback(text) {
  const l=text.toLowerCase();
  const type=l.includes('wts')||l.includes('selling')||l.includes('for sale')?'WT
  const pm=text.match(/(\d[\d,]{2,})/);
  const isUsed=l.includes('used')||l.includes('grade');
  const gm=text.match(/grade\s*([ABC][+]?)/i);
  return { type, condition:isUsed?'Used':'Brand New', model:null, storage:null, r
}

function topN(obj, n) {
  return Object.entries(obj).sort((a,b)=>b[1]-a[1]).slice(0,n).reduce((acc,[k,v])
}

// ── CHANNEL MANAGEMENT API ──

// Get all channels
app.get('/api/channels', authMiddleware, (req, res) => {
  res.json(Object.values(channels));
});

// Add / update a channel
app.post('/api/channels', authMiddleware, (req, res) => {
  const { id, label, token } = req.body;
  if (!id) return res.status(400).json({ error: 'Channel id required' });
  const ch = getOrCreateChannel(id, label);
  if (label) ch.label = label;
  if (token) ch.token = token;
  ch.active = true;
  res.json({ ok: true, channel: ch });
});

// Toggle channel active/inactive
app.patch('/api/channels/:id/toggle', authMiddleware, (req, res) => {
  const ch = channels[req.params.id];
  if (!ch) return res.status(404).json({ error: 'Channel not found' });
  ch.active = !ch.active;
  res.json({ ok: true, active: ch.active });
```

```
});

// Delete a channel
app.delete('/api/channels/:id', authMiddleware, (req, res) => {
  delete channels[req.params.id];
  res.json({ ok: true });
});

// Fetch groups from Whapi for a channel
app.get('/api/channels/:id/groups', authMiddleware, async (req, res) => {
  const ch = channels[req.params.id];
  const token = ch?.token || req.headers['x-whapi-token'];
  if (!token) return res.status(400).json({ error: 'No token for this channel' })
  try {
    const r = await fetch('https://gate.whapi.cloud/groups?count=100', {
      headers: { 'Authorization': 'Bearer ' + token }
    });
    const data = await r.json();
    const groups = (data.groups || []).map(g => ({
      id: g.id,
      name: g.name || g.subject || g.id,
      participants: g.participants_count || 0,
    }));
    // Merge into channel groups
    if (ch) {
      groups.forEach(g => {
        if (!ch.groups[g.id]) ch.groups[g.id] = { id: g.id, name: g.name, enabled
        else { ch.groups[g.id].name = g.name; ch.groups[g.id].participants = g.pa
      });
    }
    res.json(groups);
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});

// Toggle a group on/off for a channel
app.patch('/api/channels/:channelId/groups/:groupId', authMiddleware, (req, res)
  const ch = channels[req.params.channelId];
  if (!ch) return res.status(404).json({ error: 'Channel not found' });
  if (!ch.groups[req.params.groupId]) return res.status(404).json({ error: 'Group
  ch.groups[req.params.groupId].enabled = req.body.enabled;
  res.json({ ok: true });
});

// Bulk update groups for a channel
app.put('/api/channels/:channelId/groups', authMiddleware, (req, res) => {
```

```
  const ch = channels[req.params.channelId];
  if (!ch) return res.status(404).json({ error: 'Channel not found' });
  const { groups } = req.body; // [{id, enabled}]
  groups.forEach(g => { if (ch.groups[g.id]) ch.groups[g.id].enabled = g.enabled;
  res.json({ ok: true });
});

// ── DAILY SNAPSHOT SCHEDULER ──
setInterval(async () => {
  const now = new Date();
  if (now.getHours() === 23 && now.getMinutes() < 5) {
    const all = await DB.getListings({}, 2000);
    const models={};
    all.forEach(l=>{ if(l.model)models[l.model]=(models[l.model]||0)+1; });
    await DB.saveSnapshot({ date: now, totalListings: all.length, wts: all.filter
    console.log('📸 Daily snapshot saved');
  }
}, 5 * 60 * 1000);

// ── START ──
connectMongo().then(() => {
  app.listen(PORT, () => {
    console.log(`✅ WA Trading Platform on port ${PORT}`);
    console.log(`🗄️  Storage: ${db ? 'MongoDB' : 'In-Memory'}`);
    console.log(`🤖 Claude AI: ${CLAUDE_KEY ? 'Enabled' : 'Disabled'}`);
  });
});
```